SIMATS ENGINEERING

OPERATING SYSTEMS (CSA04)

ASSIGNMENT 1 – CO1

Source Code and Output of the Executed Programs

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital

Student name	MUKESH V
Register number	192524514
Submission date	06 November 2026
GitHub repository name	CSA04-192524514-Assignment1
Compiler	gcc 13.3.0
Operating system	Ubuntu 24.04.5 LTS (Linux kernel 6.18)
Compile command	gcc -std=c11 -Wall -Wextra -pthread -o .c
Tracing tool	strace 6.8

This document contains the source code and captured output of every program executed for Tasks 5, 6, 7 and 10 of Assignment 1. It accompanies the assignment report (CSA04_Assignment1_CO1_Report).

1. Task 5 – rd.c

Reads a 10,000-byte file with read() and with fgetc(); traced with strace.

Source file: Task05/rd.c

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
int main(void) {
    FILE *f = fopen("t.dat", "w");
    for (int i = 0; i < 10000; i++)
        fputc('A' + i % 26, f);
    fclose(f);
    char b[4096];
    int fd = open("t.dat", O_RDONLY);
    long s = 0;
    ssize_t n;
    while ((n = read(fd, b, sizeof b)) > 0)
        s += n;
    close(fd);
    FILE *g = fopen("t.dat", "r");
    int c;
    long k = 0;
    while ((c = fgetc(g)) != EOF)
        k++;
    fclose(g);
    printf("read(): %ld bytes, fgetc(): %ld bytes\n", s, k);
    unlink("t.dat");
    return 0;
}
```

Output of strace -e trace=openat,read,write,close ./rd (Task05/strace_trace.txt)

```
close(3)                                = 0
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "t.dat", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
write(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
write(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
write(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 1808) = 1808
close(3)                                = 0
openat(AT_FDCWD, "t.dat", O_RDONLY)     = 3
read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
read(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
read(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 4096) = 1808
read(3, "", 4096)                       = 0
close(3)                                = 0
openat(AT_FDCWD, "t.dat", O_RDONLY)     = 3
read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
read(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
read(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 4096) = 1808
read(3, "", 4096)                       = 0
close(3)                                = 0
write(1, "read(): 10000 bytes, fgetc(): 10"..., 42read(): 10000 bytes, fgetc(): 10000 bytes
) = 42
+++ exited with 0 +++
```

Output of strace -c ./rd (Task05/strace_c.txt)

```
read(): 10000 bytes, fgetc(): 10000 bytes
  0.00    0.000000           0         9           read
  0.00    0.000000           0         4           write
  0.00    0.000000           0         2           pread64
  0.00    0.000000           0         5           openat
100.00    0.000000           0        54         1 total
```

2. Task 6 – copy.c (completed program)

Copies a file with read() and write() using different buffer sizes. TODO 1 is completed.

Source file: Task06/copy.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    while (1) {
        n = read(in, buf, bufsize);
        calls++;
        if (n <= 0) break;
        if (write(out, buf, (size_t)n) != n) { calls = -1; break; }
    }
    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w"); /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);
    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Program output (matches the expected output) (Task06/output.txt)

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Output of strace -c -e trace=read,write,openat ./copy (Task06/strace_c.txt)

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 50.99    0.005958           0     10027           write
 48.46    0.005663           0     10027           read
  0.55    0.000064           7         9           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.011685           0     20063           total
```

3. Task 6 and Task 10 – bigcopy.c

Copies a 10 MB file with buffer sizes from 1 byte to 1 MB and times each.

Source file: Task10/bigcopy.c

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e3 + t.tv_nsec / 1e6;
}
long copy(const char *s, const char *d, size_t b) {
    char *buf = malloc(b);
    int in = open(s, O_RDONLY),
        out = open(d, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    long c = 0;
    ssize_t n;
    while ((n = read(in, buf, b)) > 0) {
        c++;
        write(out, buf, (size_t)n);
    }
    close(in);
    close(out);
    free(buf);
    return c;
}
int main(void) {
    FILE *f = fopen("big.dat", "w");
    char blk[4096];
    for (int i = 0; i < 4096; i++)
        blk[i] = 'A' + i % 26;
    for (int i = 0; i < 2560; i++)
        fwrite(blk, 1, 4096, f);
    fclose(f); /* 10 MB */
    size_t sz[] = {1, 64, 512, 4096, 65536, 1048576};
    printf("%-10s %-12s %-10s\n", "buffer", "read calls", "time ms");
    for (int i = 0; i < 6; i++) {
        double t = now();
        long c = copy("big.dat", "big.out", sz[i]);
        printf("%-10zu %-12ld %-10.1f\n", sz[i], c, now() - t);
    }
    unlink("big.dat");
    unlink("big.out");
    return 0;
}
```

Output (buffer size, read calls, time in ms) (Task10/bigcopy_out.txt)

```
buffer     read calls   time ms
1          10485760     5209.1
64         163840       82.2
512        20480        28.0
4096       2560         14.1
65536      160          9.2
1048576    10           8.5
```

4. Task 7 – shell.c (completed program)

A small shell that runs two jobs with fork(), execlp() and waitpid(). TODO 1 is completed.

Source file: Task07/shell.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int status;
    pid_t pid;
    printf("shell: running job 1\n");
    fflush(stdout); /* empty the buffer before fork() */

    pid = fork();
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);
    }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3); /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Program output (matches the expected output) (Task07/output.txt)

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

5. Task 7 – shell_variant.c

Variant with the second fflush(stdout) removed and exit(3) used instead of _exit(3).

Source file: Task07/shell_variant.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int status;
    pid_t pid;
    printf("shell: running job 1\n");
    fflush(stdout); /* empty the buffer before fork() */

    pid = fork();
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);
    }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    pid = fork();
    if (pid == 0) exit(3); /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Output when redirected to a file: ./shell_variant > variant_out.txt (Task07/variant_out.txt)

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

6. Task 7 and Task 10 – spawn2.c

Starts 1,000 commands with fork+exec and with posix_spawn.

Source file: Task10/spawn2.c

```
#define _POSIX_C_SOURCE 200809L
#include <spawn.h>
#include <stdio.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
extern char **environ;
static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e3 + t.tv_nsec / 1e6;
}
int main(void) {
    double t0 = now();
    for (int i = 0; i < 1000; i++) {
        pid_t p = fork();
        if (p == 0) {
            execl("/bin/true", "true", (char *)NULL);
            _exit(127);
        }
        waitpid(p, NULL, 0);
    }
    double t1 = now();
    for (int i = 0; i < 1000; i++) {
        pid_t p;
        char *a[] = {"true", NULL};
        posix_spawn(&p, "/bin/true", NULL, NULL, a, environ);
        waitpid(p, NULL, 0);
    }
    double t2 = now();
    printf("1000 x fork+exec : %.1f ms\n1000 x posix_spawn: %.1f ms\n",
           t1 - t0, t2 - t1);
    return 0;
}
```

Output (3 runs) (Task10/spawn2_out.txt)

```
1000 x fork+exec : 904.2 ms
1000 x posix_spawn: 647.9 ms
1000 x fork+exec : 706.5 ms
1000 x posix_spawn: 671.9 ms
1000 x fork+exec : 684.3 ms
1000 x posix_spawn: 672.6 ms
```

7. Task 10 – spawn.c

Creates 1,000 processes with fork() and 1,000 threads with pthread_create().

Source file: Task10/spawn.c

```
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
#define N 1000
static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e3 + t.tv_nsec / 1e6;
}
static void *work(void *a) {
    (void)a;
    return NULL;
}
int main(void) {
    double t0 = now();
    for (int i = 0; i < N; i++) {
        pid_t p = fork();
        if (p == 0)
            _exit(0);
        waitpid(p, NULL, 0);
    }
    double t1 = now();
    for (int i = 0; i < N; i++) {
        pthread_t t;
        pthread_create(&t, NULL, work, NULL);
        pthread_join(t, NULL);
    }
    double t2 = now();
    printf("1000 processes (fork+wait): %.2f ms\n1000 threads (create+join): "
           "%.2f ms\n",
           t1 - t0, t2 - t1);
    return 0;
}
```

Output (3 runs) (Task10/spawn_out.txt)

```
1000 processes (fork+wait): 50.11 ms
1000 threads (create+join): 8.87 ms
1000 processes (fork+wait): 47.17 ms
1000 threads (create+join): 8.22 ms
1000 processes (fork+wait): 47.43 ms
1000 threads (create+join): 8.86 ms
```

8. Task 10 – mem.c

Measures resident memory with 1,000 threads alive.

Source file: Task10/mem.c

```
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
static void *w(void *a) {
   (void)a;
   sleep(1);
   return NULL;
}
static long rss(void) {
   FILE *f = fopen("/proc/self/status", "r");
   char l[128];
   long v = 0;
   while (fgets(l, 128, f))
      if (!strncmp(l, "VmRSS:", 6))
         sscanf(l + 6, "%ld", &v);
   fclose(f);
   return v;
}
int main(void) {
   long r0 = rss();
   pthread_t t[1000];
   for (int i = 0; i < 1000; i++)
      pthread_create(&t[i], NULL, w, NULL);
   printf("RSS before %ld kB, with 1000 threads %ld kB (increase %ld kB)\n",
         r0, rss(), rss() - r0);
   for (int i = 0; i < 1000; i++)
      pthread_join(t[i], NULL);
   return 0;
}
```

Output (Task10/mem_out.txt)

```
RSS before 1396 kB, with 1000 threads 10228 kB (increase 8832 kB)
```